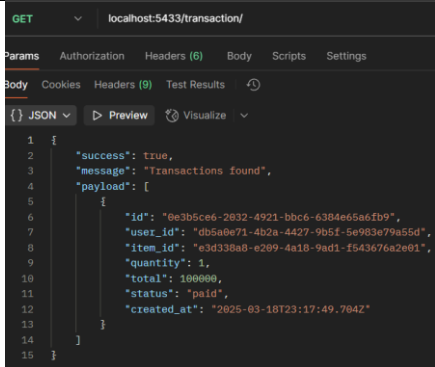
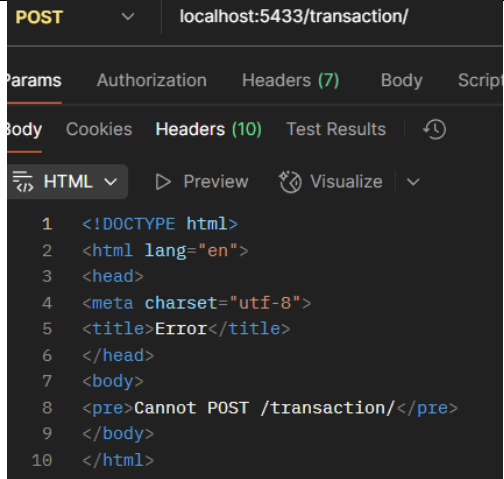


PRAKTIKUM SISTEM BASIS DATA

Nama	Wesley Frederick Oh	No. Modul	6
NPM	2306202763	Tipe	Tugas Tambahan

1. Get All Transaction

Method	Endpoint	Success	Failed
GET	/transaction		

2. Code Upgrades

Tipe	Implementasi	Deskripsi
Security	<pre> PORT = 5433 PG_CONNECTION_STRING = postgresql://express_wesley_own sslmode=require CLOUDINARY_CLOUD_NAME = dp1jezm2q CLOUDINARY_API_KEY = 668728787957176 CLOUDINARY_API_SECRET = pq1Zz-U8gCwX1bjajZolUvcAj-E </pre>	Menaruh Cloudinary Cloud Name, API Key, dan API Secret di .env agar lebih mudah ditemukan dan lebih mudah di debug.

	<pre>const dotenv = require("dotenv"); dotenv.config(); cloudinary.config({ cloud_name: process.env.CLOUDINARY_CLOUD_NAME, api_key: process.env.CLOUDINARY_API_KEY, api_secret: process.env.CLOUDINARY_API_SECRET, secure: true });</pre>	
Scalability	✓ src ✓ controllers - JS itemcontroller.js - JS storecontroller.js - JS transactioncontroller.js - JS usercontroller.js ✓ routes - JS itemroute.js - JS routes.js - JS transactionroute.js - JS userroute.js ✓ utils - JS connect.js	Memisahkan controllers.js menjadi itemcontroller, storecontroller, transactioncontroller, dan usercontroller, ditaruh di folder controllers. Begitupun semua route ditaruh di folder routes, dan connect.js di folder utils.

BONUS

1. Token Github

Github Token

.....

You can create a personal access token [here](#). In the scopes section you should check `repo`.

Save

2. Dockerfile untuk express backend

```
FROM node
WORKDIR /app
COPY package.json /app
RUN npm install
COPY . /app
CMD ["node", "index.js"]
```

3. Push ke github repo private

```
PS D:\Praktikum\SBD\Modul 5\CS> git add .
PS D:\Praktikum\SBD\Modul 5\CS> git commit -m "dockers"
[master 2c50988] dockers
 4 files changed, 23 insertions(+), 2 deletions(-)
 create mode 100644 .dockerignore
 create mode 100644 dockerfile
PS D:\Praktikum\SBD\Modul 5\CS> git push origin master
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 20 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 967 bytes | 322.00 KiB/s, done.
Total 8 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/sleepingpolice-afk/CS_SBD.git
 d2c263b..2c50988 master -> master
PS D:\Praktikum\SBD\Modul 5\CS> |
```

4. App service

Create App Service

Service Name

express_wesleyfrederickoh

Create

5. Service Github

sbd / express_wesleyfrederickoh APP + Service Templates ⋮

Source

Upload **Github** Docker Image Git Dockerfile

Owner * **Repository ***

Github username or organization name Github repository name

i github.com / sleepingpolice-afk / CS_SBD

Branch * **Build Path ***

This has to be a valid branch in your repo This is useful if you have a monorepo

Save

6. sd

7. env

Environment Variables

```

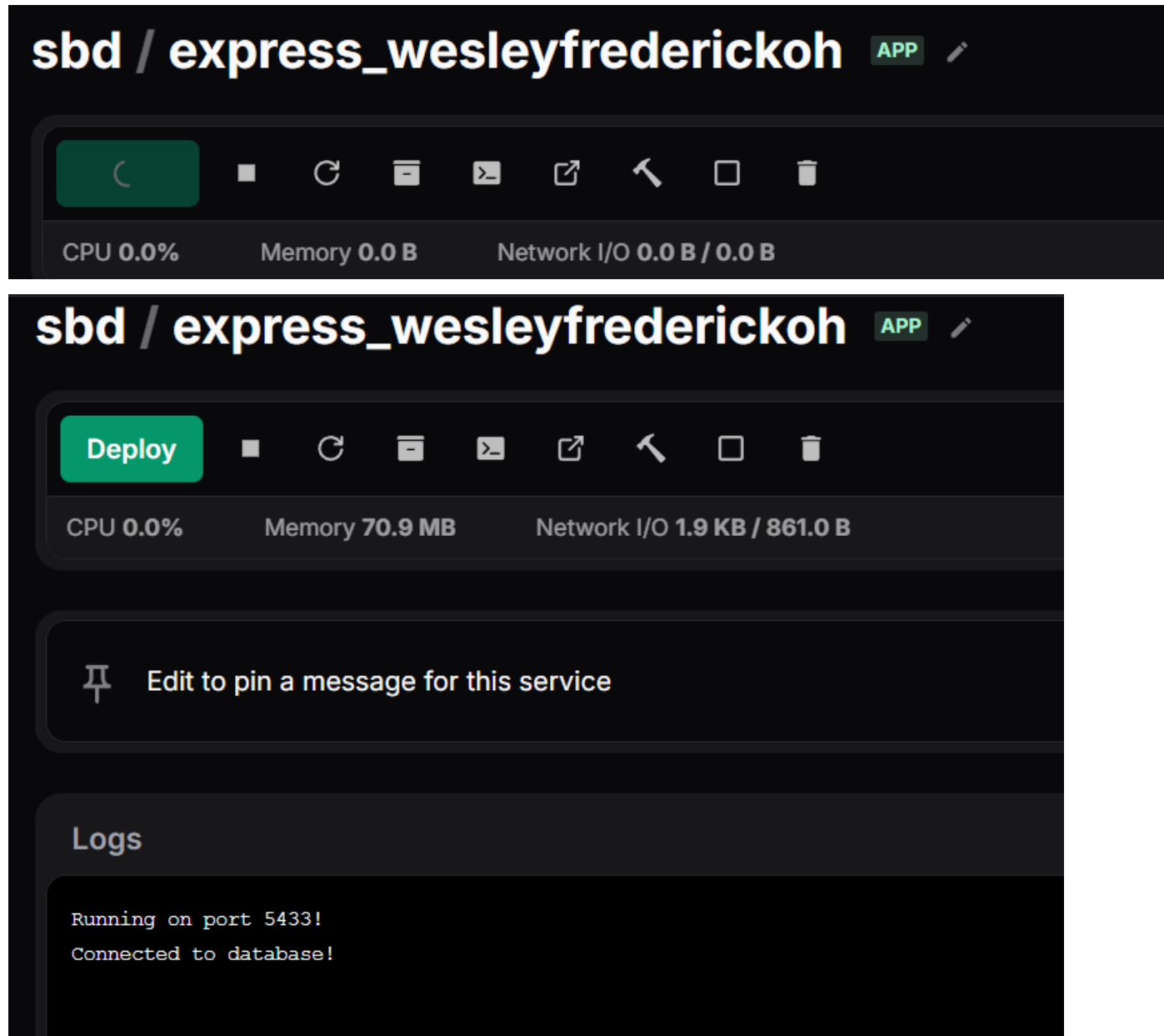
1  PORT = 5433
2
3  #NEONTECH
4  #PG_CONNECTION_STRING = postgresql://express_wesley_owner:npg_D1npoFwxSE5M@ep-floral-fire-a51xbu81.us-east-2.aws.neon.tech/express
5
6
7  #ALIBABA
8  PG_CONNECTION_STRING = postgres://wesley_sbd2:wes110h0h@8.215.196.171:5433/WesleyFrederick0h_2?sslmode=disable
9
10 CLOUDINARY_CLOUD_NAME = dp1jezm2q
11 CLOUDINARY_API_KEY = 668728787957176
12 CLOUDINARY_API_SECRET = pq1Zz-U8gCwX1bjajZolUvcAj-E

```

☐ Create .env file

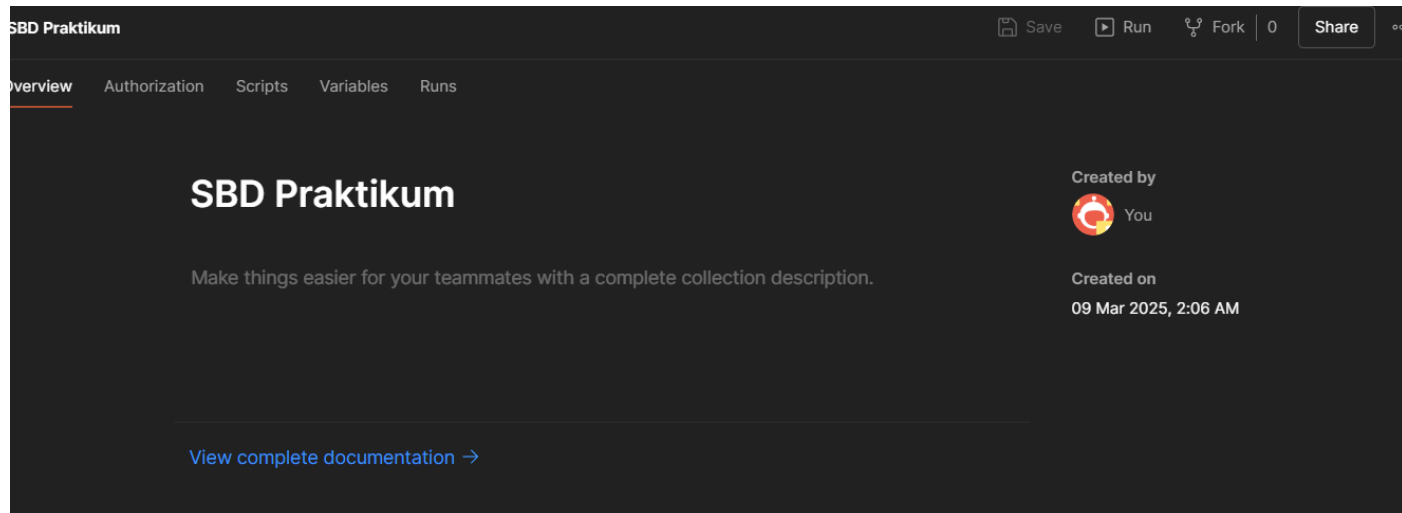
Save

8. Deploy

The screenshot shows the Netlab interface for a service named 'sbd / express_wesleyfrederickoh'. The top bar is dark blue with the service name and a green 'APP' button. Below this is a toolbar with icons for various actions. The status bar shows 'CPU 0.0%', 'Memory 0.0 B', and 'Network I/O 0.0 B / 0.0 B'. The main content area has a dark blue background with a 'Deploy' button, a toolbar, and a status bar showing 'CPU 0.0%', 'Memory 70.9 MB', and 'Network I/O 1.9 KB / 861.0 B'. Below the status bar is a section for 'Edit to pin a message for this service' and a 'Logs' section showing the output: 'Running on port 5433!' and 'Connected to database!'.

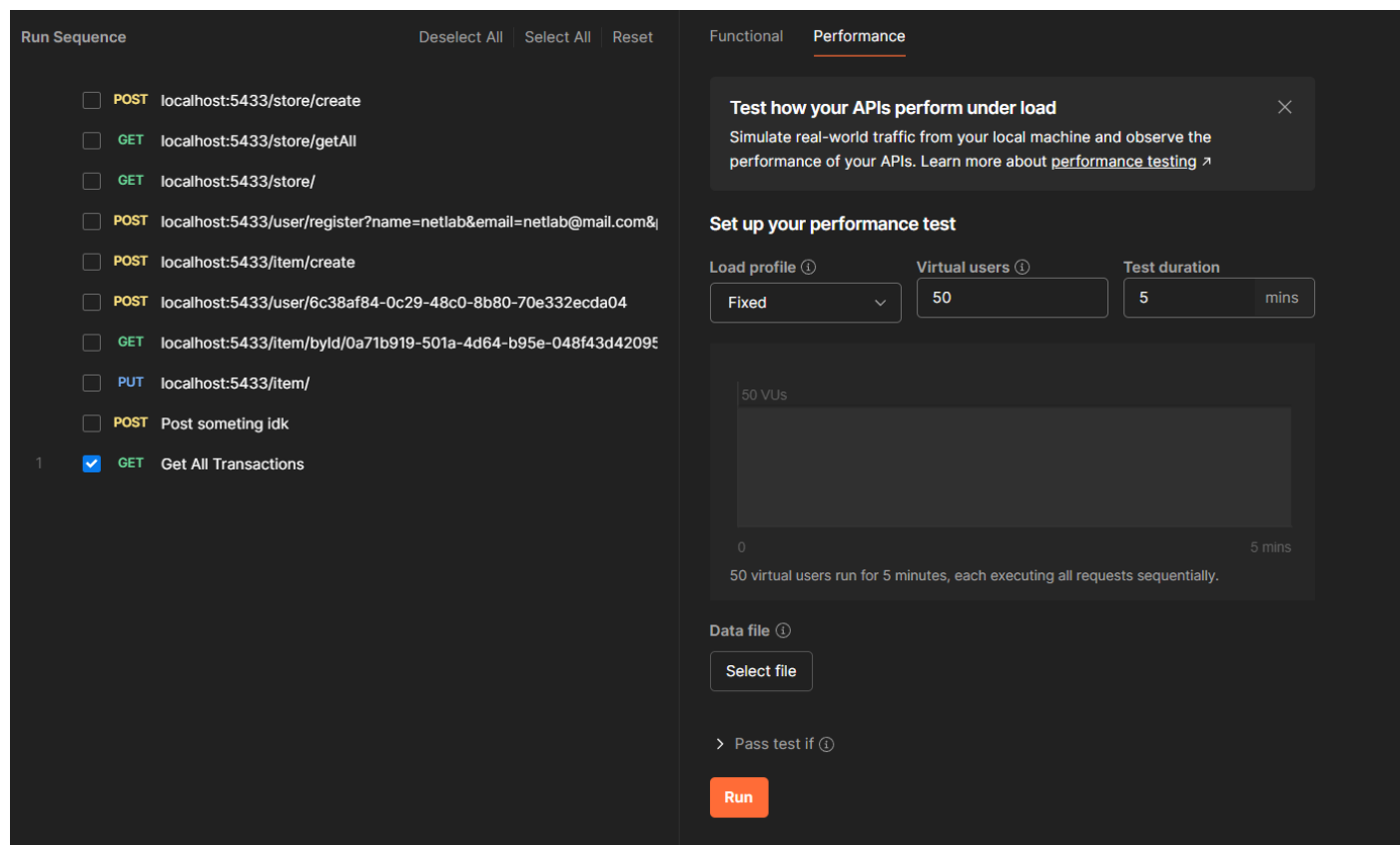
service telah berjalan

9. Memasukkan dummybonustutam.sql
10. Stress test



Domain: <https://sbd-express-wesleyfrederickoh.q7szht.easypanel.host/>

11. Test 50 User selama 5 menit stress test



12. Hasil Stress Test

Akan dilampirkan di bagian akhir pdf ini juga.

13. Bagian yang menandakan bahwa stress test berhasil adalah pada bagian metric-nya. Terdapat beberapa metric yang digunakan, seperti Total requests, request per detik, Minimum response time, average response time, response time untuk persentil ke-90(?), maksimum response time, serta persentase error yang terjadi dari semua total request.

Metric yang menandakan tes ini berhasil adalah bagian errornya yang sangat rendah, yaitu sekitar 2.5% saja dari total request sebanyak 1800 lebih yang diberikan dalam 5 menit. Jadi bisa dipastikan server API bisa handle banyak traffic sekaligus dengan persentase error yang cukup rendah. Namun, balik lagi tergantung tipe aplikasi yang digunakan, jika aplikasi tidak mentolerir error sekecil apapun, maka 2.5% mungkin termasuk error yang tinggi.

Error di sini yang tercatat adalah ESOCKETTIMEOUT, yang artinya percobaan untuk terhubung ke server telah timed out karena tidak menerima response dalam waktu tertentu.